

LAB HANDOUT

Exploratory Data Analysis with Python

Understanding a Dataset through Statistics and Visualization

Dataset	Titanic — the same dataset from the Data Preprocessing lab
Prerequisite	Lab 2: Data Preprocessing (missing values, encoding, scaling)
Tools	Python 3, Jupyter Notebook / Google Colab / VS Code
Libraries	pandas, numpy, seaborn, matplotlib
Mode	Hands-on: copy each block, run it, and study the output

1. How This Lab Connects to Lab 2

In Lab 2 (Data Preprocessing) you learned to CLEAN a dataset: fill missing values, encode text into numbers, scale features, handle outliers, and split the data. This lab is the natural next step. Before building any model, a data scientist first EXPLORES the cleaned data to understand it. This is called Exploratory Data Analysis (EDA).

The workflow is: Raw data → Preprocessing → EDA (this lab) → Modelling. We reuse the exact same Titanic dataset, so you already know the columns. We even begin by repeating the two most important cleaning steps from Part 1 so the data is ready.

What is EDA?: Exploratory Data Analysis is the process of summarizing a dataset's main characteristics using statistics and visualization to discover patterns, spot anomalies, test assumptions, and form questions BEFORE modelling. In short: we let the data tell its story.

2. Learning Objectives

By the end of this lab, you will be able to:

- Compute and interpret descriptive statistics for numeric and categorical columns.
- Perform univariate analysis : study one variable at a time with histograms, box plots and count plots.
- Perform bivariate analysis: study relationships between two variables (e.g. survival vs gender).
- Build and read a correlation heatmap to find which features relate to the target.
- Translate charts and numbers into clear, written insights about the data.

3. Step 1: Setup & Preprocessing Recap

Load the Titanic data and re-apply the key cleaning steps from Part 1.

We load the dataset and immediately fill the missing age and embarked values — exactly as in Part 1 — so EDA runs on clean data.

```
# =====
# STEP 1 - Setup and preprocessing recap (from Part 1)
# =====
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style('whitegrid')          # nicer looking charts
pd.set_option('display.width', 100)
pd.set_option('display.max_columns', 20)

cols = ['survived', 'pclass', 'sex', 'age',
```

```

    'sibsp', 'parch', 'fare', 'embarked']
df = sns.load_dataset('titanic')[cols].copy()

# --- recap of Part 1: handle missing values ---
df['age'] = df['age'].fillna(df['age'].median())
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])

print("Shape:", df.shape)
print("Missing values remaining:", df.isnull().sum().sum())
print(df.head())

```

EXPECTED OUTPUT

```

Shape: (891, 8)
Missing values remaining: 0
  survived  pclass    sex   age  sibsp  parch   fare embarked
0         0       3   male  22.0     1     0   7.2500         S
1         1       1  female  38.0     1     0  71.2833         C
2         1       3  female  26.0     0     0   7.9250         S
3         1       1  female  35.0     1     0  53.1000         S
4         0       3   male  35.0     0     0   8.0500         S

```

Notice: This is the same cleaning you did in Lab 2, compressed into two lines. With zero missing values, every chart and statistic below will be reliable.

4. Step 2: Descriptive Statistics

Summarize numeric and categorical columns numerically.

2.1 Numeric summary

`describe()` gives count, mean, standard deviation, min, max and the quartiles for every numeric column. It is the fastest way to grasp the scale and spread of your data.

```
print(df.describe())
```

EXPECTED OUTPUT

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	891.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.361582	0.523008	0.381594	32.204208
std	0.486592	0.836071	13.019697	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	22.000000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	35.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Quick reading: the average passenger was about 29 years old; the mean of survived (0.38) tells us only ~38% survived; and fare has a huge spread (std 49.7, max 512) which gives a hint of skew we will see visually in Step 3.

2.2 Categorical summary

For text columns, `describe(include='object')` reports how many unique values exist and which is most frequent.

```
print(df.describe(include='object'))
```

EXPECTED OUTPUT

	sex	embarked
count	891	891
unique	2	3
top	male	S
freq	577	646

There are 577 males (the majority) and Southampton (S) was the most common boarding port, used by 646 of 891 passengers.

2.3 Counting categories with `value_counts()`

```

print("Survived:\n", df['survived'].value_counts())
print("\nSurvival rate: %.3f" % df['survived'].mean())

```

```
print("\nPclass:\n", df['pclass'].value_counts().sort_index())
print("\nEmbarked:\n", df['embarked'].value_counts())
```

EXPECTED OUTPUT

```
Survived:
  survived
0      549
1      342
Name: count, dtype: int64

Survival rate: 0.384

Pclass:
  pclass
1      216
2      184
3      491
Name: count, dtype: int64

Embarked:
  embarked
S      646
C      168
Q       77
Name: count, dtype: int64
```

Insight: 549 died vs 342 survived : the classes are imbalanced (recall this idea from Lab 2). Third class was by far the largest group with 491 passengers.

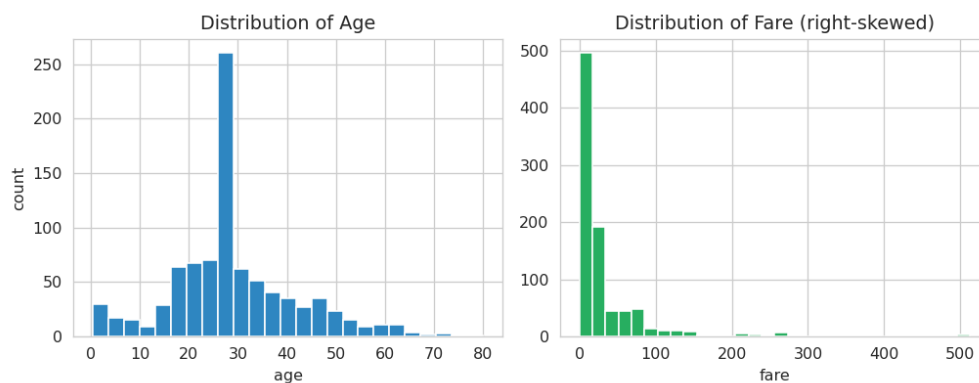
5. Step 3: Univariate Analysis: Numeric Columns

"Univariate" means one variable. We look at the shape (distribution) of each numeric column on its own.

3.1 Histograms : the shape of a distribution

```
fig, ax = plt.subplots(1, 2, figsize=(9, 3.6))
ax[0].hist(df['age'], bins=25, color='#2E86C1', edgecolor='white')
ax[0].set_title('Distribution of Age')
ax[0].set_xlabel('age'); ax[0].set_ylabel('count')
ax[1].hist(df['fare'], bins=30, color='#27ae60', edgecolor='white')
ax[1].set_title('Distribution of Fare (right-skewed)')
ax[1].set_xlabel('fare')
plt.tight_layout()
plt.show()
```

EXPECTED OUTPUT (CHARTS LIKE THESE)



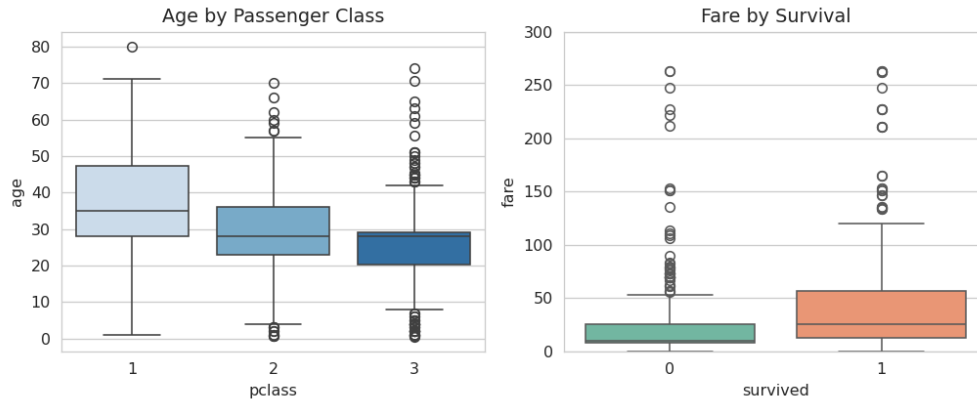
Age is roughly bell-shaped and centred near 28–30. Fare is strongly right-skewed , most tickets were cheap, with a long tail of expensive fares. (Recall from Lab 2: a right-skew like this is exactly what a log transform would reduce.)

3.2 Box plots : spread and outliers

A box plot shows the median, the quartiles (the box), and outliers (the dots). It is perfect for comparing a numeric variable across categories.

```
fig, ax = plt.subplots(1, 2, figsize=(9, 3.8))
sns.boxplot(data=df, x='pclass', y='age', ax=ax[0], palette='Blues')
ax[0].set_title('Age by Passenger Class')
sns.boxplot(data=df, x='survived', y='fare', ax=ax[1], palette='Set2')
ax[1].set_title('Fare by Survival')
ax[1].set_ylim(0, 300)
plt.tight_layout()
plt.show()
```

EXPECTED OUTPUT (CHARTS LIKE THESE)



Insight: First-class passengers were older on average than third-class. And survivors (right box) tended to pay higher fares. This gives us an early clue that wealth relates to survival.

6. Step 4 — Univariate Analysis: Categorical Columns

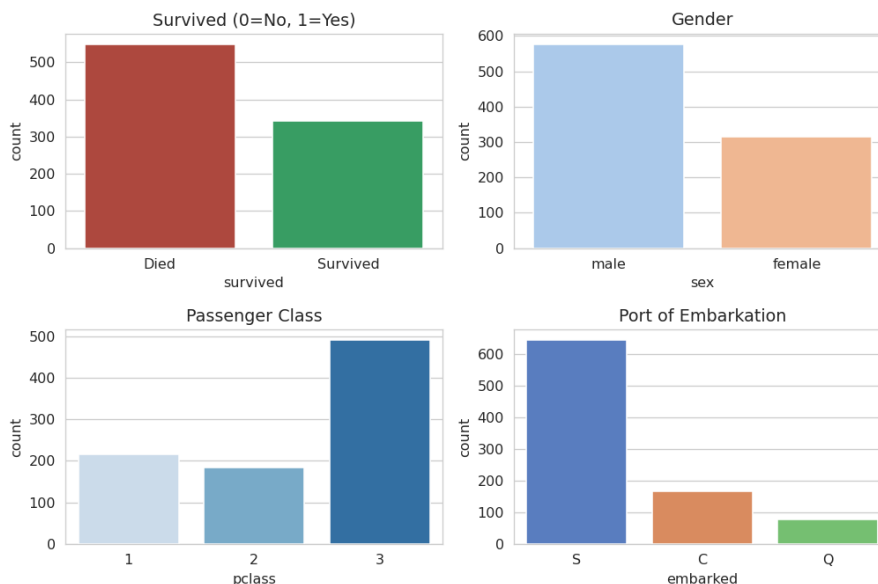
15 min

Count plots for categorical variables.

For categorical columns we count how many rows fall in each category using a count plot (a bar chart of frequencies).

```
fig, ax = plt.subplots(2, 2, figsize=(9, 6))
sns.countplot(data=df, x='survived', ax=ax[0,0],
              palette=['#c0392b', '#27ae60'])
ax[0,0].set_title('Survived (0=No, 1=Yes)')
sns.countplot(data=df, x='sex', ax=ax[0,1], palette='pastel')
ax[0,1].set_title('Gender')
sns.countplot(data=df, x='pclass', ax=ax[1,0], palette='Blues')
ax[1,0].set_title('Passenger Class')
sns.countplot(data=df, x='embarked', ax=ax[1,1], palette='muted')
ax[1,1].set_title('Port of Embarkation')
plt.tight_layout()
plt.show()
```

EXPECTED OUTPUT (FOUR CHARTS)



At a glance: more people died than survived, there were far more men than women, third class dominated, and most passengers boarded at Southampton.

7. Step 5 — Bivariate Analysis: What Affects Survival?

🕒 20 min

Relate each feature to the target using groupby and grouped bar charts.

"Bivariate" means two variables. Here we connect each feature to survived to see what mattered. `groupby()` computes the survival rate within each group.

5.1 Survival rate by a single feature

```
print("Survival rate by sex:")
print(df.groupby('sex')['survived'].mean().round(3))
print("\nSurvival rate by class:")
print(df.groupby('pclass')['survived'].mean().round(3))
print("\nSurvival rate by embarked:")
print(df.groupby('embarked')['survived'].mean().round(3))
```

EXPECTED OUTPUT

```
Survival rate by sex:
sex
female    0.742
male      0.189
Name: survived, dtype: float64
```

```
Survival rate by class:
pclass
1      0.630
2      0.473
3      0.242
Name: survived, dtype: float64
```

```
Survival rate by embarked:
embarked
C      0.554
Q      0.390
S      0.339
Name: survived, dtype: float64
```

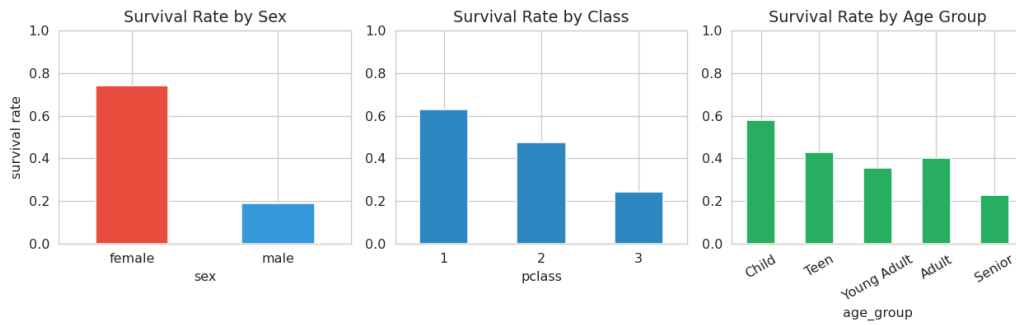
Powerful insight: 74% of women survived versus only 19% of men — the strongest single pattern in the data ("women and children first"). Survival also climbs steeply with class: 63% in first class down to 24% in third.

5.2 Visualizing survival rates

```
fig, ax = plt.subplots(1, 3, figsize=(11, 3.6))
df.groupby('sex')['survived'].mean().plot(
    kind='bar', ax=ax[0], color=['#e74c3c', '#3498db'])
ax[0].set_title('Survival Rate by Sex')
df.groupby('pclass')['survived'].mean().plot(
    kind='bar', ax=ax[1], color='#2E86C1')
ax[1].set_title('Survival Rate by Class')

# age groups via binning (recall Part 1 binning idea)
df['age_group'] = pd.cut(df['age'], bins=[0,12,18,35,60,100],
    labels=['Child','Teen','Young Adult','Adult','Senior'])
df.groupby('age_group')['survived'].mean().plot(
    kind='bar', ax=ax[2], color='#27ae60')
ax[2].set_title('Survival Rate by Age Group')
for a in ax: a.set_ylim(0, 1)
plt.tight_layout()
plt.show()
```

EXPECTED OUTPUT (THREE CHARTS)



Children survived at the highest rate among age groups, and seniors the lowest — consistent with the "children first" pattern. Note we reused the binning idea from Part 1 to create age groups.

5.3 Two features at once — sex and class together

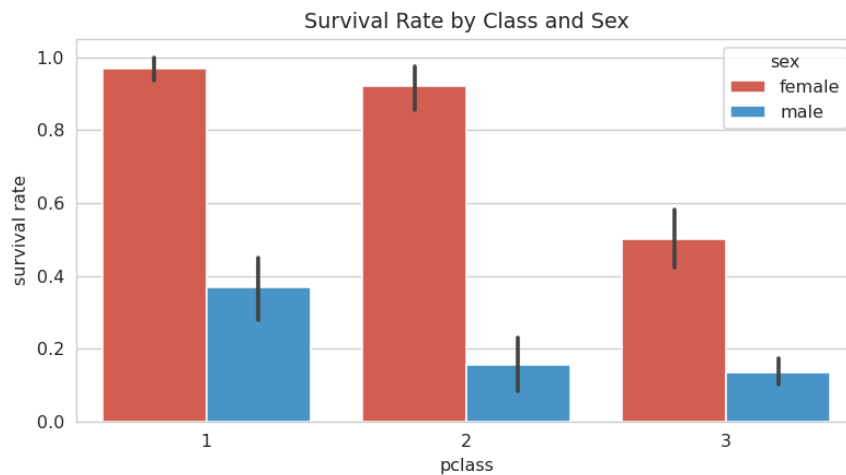
A crosstab (or grouped bar) shows how two categories combine to affect the target.

```
print(pd.crosstab(df['sex'], df['pclass'],
                  values=df['survived'], aggfunc='mean').round(3))

sns.barplot(data=df, x='pclass', y='survived', hue='sex',
            palette=['#e74c3c', '#3498db'])
plt.title('Survival Rate by Class and Sex')
plt.ylabel('survival rate')
plt.tight_layout()
plt.show()
```

EXPECTED OUTPUT

pclass	1	2	3
sex			
female	0.968	0.921	0.500
male	0.369	0.157	0.135



Key insight: Almost every first- and second-class woman survived (97% and 92%). A third-class man had only a 14% chance. Sex and class together predict survival far better than either alone.

8. Step 6 — Correlation Analysis

Measure and visualize linear relationships between numeric features.

Correlation measures how strongly two numeric variables move together, from -1 (opposite) through 0 (none) to +1 (together). We first encode sex to a number (as in Part 1) so it can join the correlation.

```
df['sex_encoded'] = df['sex'].map({'male': 0, 'female': 1})
num_cols = ['survived', 'pclass', 'sex_encoded', 'age', 'sibsp', 'parch', 'fare']

print("Correlation with survival (sorted):")
```

```
print(df[num_cols].corr()['survived'].sort_values(ascending=False).round(3))
```

EXPECTED OUTPUT

```
Correlation with survival (sorted):
survived      1.000
sex_encoded    0.543
fare          0.257
parch         0.082
sibsp        -0.035
age          -0.065
pclass       -0.338
Name: survived, dtype: float64
```

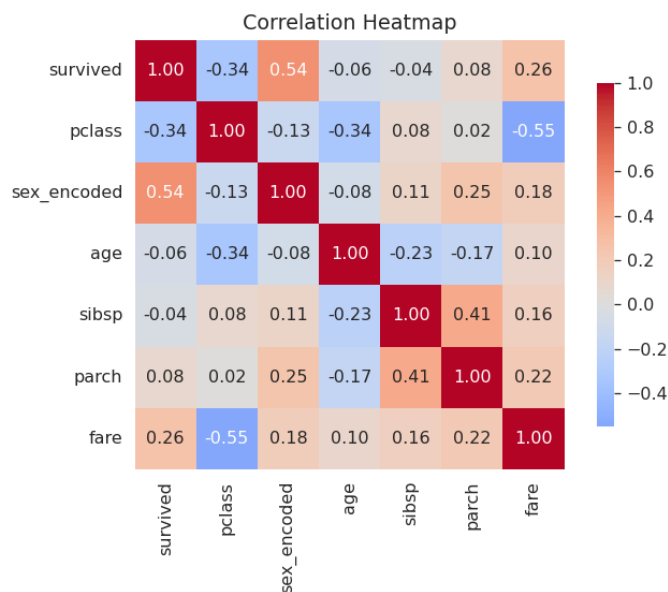
sex_encoded has the strongest positive correlation with survival (0.54; female=1), while pclass is negatively correlated (-0.34; a higher class number means lower survival). This matches everything we saw in Step 5.

6.1 The correlation heatmap

A heatmap turns the whole correlation table into colour, so patterns jump out. Red = positive, blue = negative.

```
corr = df[num_cols].corr()
plt.figure(figsize=(6.5, 5))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, square=True, cbar_kws={'shrink': 0.8})
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```

EXPECTED OUTPUT (A HEATMAP LIKE THIS)



How to read it: Scan the "survived" row/column. Strong colours (far from white) mean strong relationships. sex_encoded is the reddest, pclass the bluest — those are your most predictive features for a future model.

9. Step 7 — Summary of Insights

Good EDA ends with plain-language conclusions. From this dataset we learned:

- Gender was the single biggest factor: ~74% of women survived vs ~19% of men.
- Class mattered strongly: first-class survival (63%) was more than double third-class (24%).
- Wealth (fare) related to survival — survivors paid higher fares on average.
- Children had the best survival rate among age groups; seniors the worst.
- Combining sex and class was most revealing: 1st/2nd-class women almost all survived; 3rd-class men rarely did.
- The target is imbalanced (549 died vs 342 survived) — important to remember before modelling.

The takeaway: EDA turned a clean table of numbers into a clear story: on the Titanic, being female, wealthy, in a high class, and young dramatically raised your chance of survival. These are exactly the features a model should focus on.

10. Quick Reference — EDA Toolkit

Goal	Tool / Function
Numeric summary	<code>df.describe()</code>
Category summary	<code>df.describe(include='object'), value_counts()</code>
One numeric variable	<code>plt.hist(), sns.boxplot()</code>
One categorical variable	<code>sns.countplot()</code>
Feature vs target	<code>df.groupby(...).mean(), sns.barplot()</code>
Two categories vs target	<code>pd.crosstab(...)</code>
Relationships (numeric)	<code>df.corr(), sns.heatmap()</code>

11. Lab Exercises

Use the same df. Write your code, run it, and paste the output or chart.

1. Plot a histogram of the sibsp column. Is it skewed? Describe its shape in one sentence.
2. Compute the survival rate grouped by embarked AND sex together using `pd.crosstab()`. Which combination had the highest survival rate?
3. Make a box plot of age split by survived. Did survivors tend to be younger or older?
4. Create a count plot of your `age_group` column. Which age group was the largest?
5. Add a new feature `family_size = sibsp + parch + 1`. Compute the survival rate by `family_size`. Is there a family size that survived best?
6. Challenge: from the correlation results, name the two features most and least useful for predicting survival, and justify your answer in two sentences.

End of Lab Handout — Exploratory Data Analysis with Python (Part 2 of the Titanic series).